COMPSCI 689
Lecture 23: Advanced Experiment Designs

Benjamin M. Marlin

College of Information and Computer Sciences
University of Massachusetts Amherst

Slides by Benjamin M. Marlin (marlin@cs.umass.edu).

Review

- To date we have discussed basic error metrics and loss functions for supervised and unsupervised learning.
- We have also discussed two basic machine learning experiment designs: the train-test experiment and the train-validation-test experiment.
- In this lecture, we'll introduce additional performance measures and more complex experiment types.

Error, Accuracy

- Classification Error Rate (E): Number of incorrectly classified instances over the data set size.

$$E = \frac{1}{N} \sum_{n=1}^{N} [y_n \neq f_\theta(\mathbf{x}_n)]$$

- Classification Accuracy Rate (A): Number of correctly classified instances over the data set size.

$$A = \frac{1}{N} \sum_{n=1}^{N} [y_n = f_\theta(\mathbf{x}_n)]$$

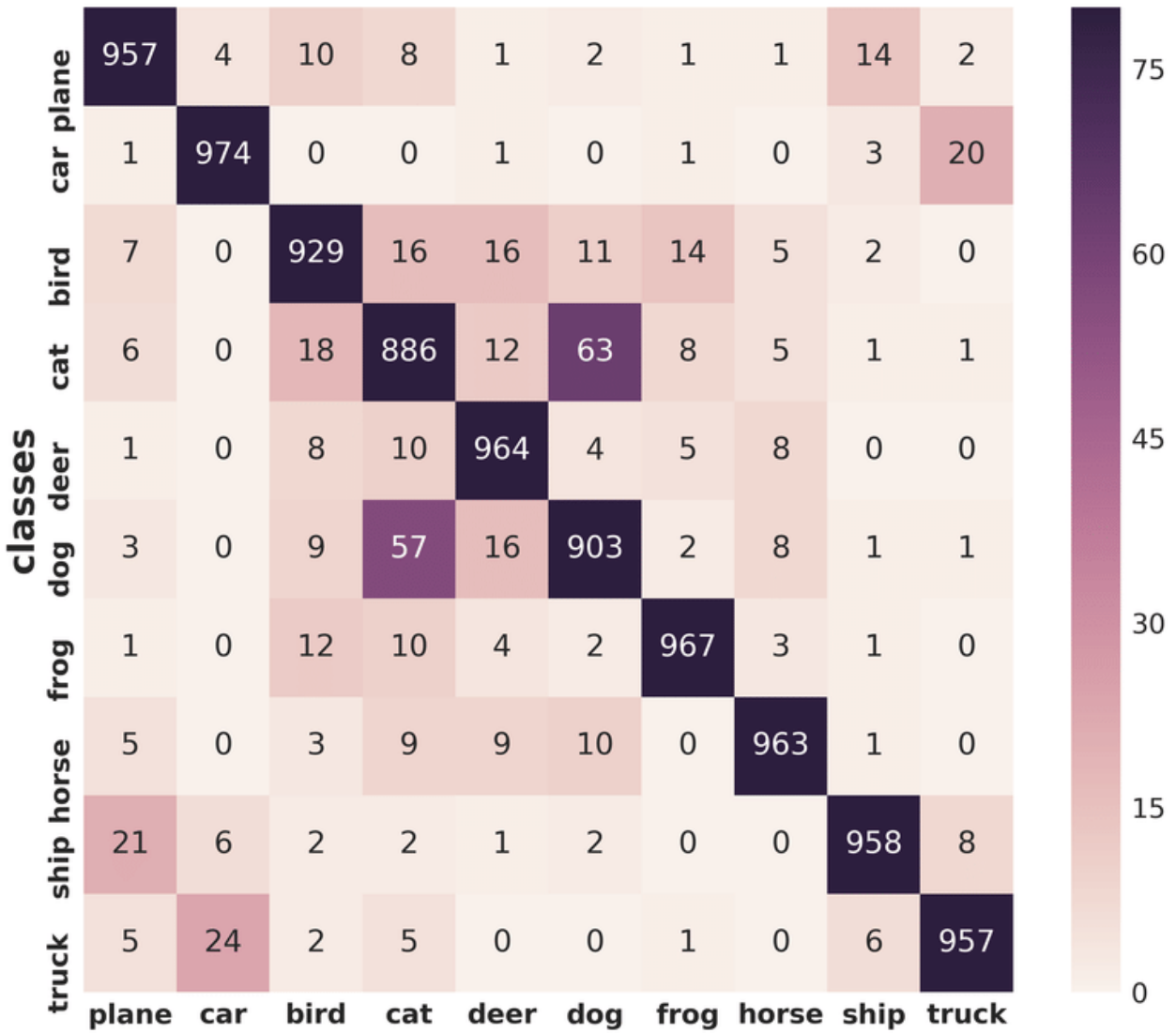
Confusion Matrix

- A confusion matrix M is a representation of the number of instances where the true class y and the class predicted by the model is y'.
- The entries in the confusion matrix are computed using:

$$M_{y,y'} = \sum_{n=1}^{N} [y_n = y][f_\theta(\mathbf{x}_n) = y']$$

- The accuracy rate is equal to the sum of the diagonal entries of M divided by the number of test instances.
- The error rate is equal to the sum of the off-diagonal entries of M divided by the number of test instances.

Example: CIFAR10



Weighted Measures

- Class-Weighted Classification Error Rate (Ew): Number of incorrectly classified instances over the data set size.

$$Ew = \frac{\sum_{n=1}^{N} w_{y_n}[y_n \neq f_\theta(\mathbf{x}_n)]}{\sum_{n'=1}^{N} w_{y_{n'}}}$$

- Misclassification Cost (MC): Avergage over test instances of the misclassification cost based on cost matrix C.

$$MC = \frac{1}{N}\sum_{n=1}^{N} C[y_n, f_\theta(\mathbf{x}_n)]$$

Performance Measures for Binary Classification

Binary Classification Confusion Matrix

	Positive Prediction	Negative Prediction
Positive Class	# True Positives (TP)	# False Negatives (FN)
Negative Class	# False Positives (FP)	# True Negatives (TN)

Performance Measures for Binary Classification

- Precision (P): The fraction of true positives to total positive predictions.

$$P = \frac{\sum_{n=1}^{N} [y_n = 1][f_\theta(\mathbf{x}_n) = 1]}{\sum_{n=1}^{N} [f_\theta(\mathbf{x}_n) = 1]} = \frac{TP}{TP + FP}$$

- Recall (R): The fraction of true positives to total positives instances.

$$R = \frac{\sum_{n=1}^{N} [y_n = 1][f_\theta(\mathbf{x}_n) = 1]}{\sum_{n=1}^{N} [y_n = 1]} = \frac{TP}{TP + FN}$$

- F1 Score: $2(P \cdot R)/(P + R)$.

Performance Measures for Binary Classification

- True Positive Rate (TPR): The fraction of true positives to total positives instances.

$$TPR = \frac{\sum_{n=1}^{N} [y_n = 1][f_\theta(\mathbf{x}_n) = 1]}{\sum_{n=1}^{N} [y_n = 1]} = \frac{TP}{TP + FN}$$

- False Positive Rate (FPR): The fraction of false positives to total negative instances.

$$FPR = \frac{\sum_{n=1}^{N} [y_n \neq 1][f_\theta(\mathbf{x}_n) = 1]}{\sum_{n=1}^{N} [y_n \neq 1]} = \frac{FP}{FP + TN}$$

Performance Measures for Binary Classification

- Suppose we have access to a discriminant function $g_\theta(\mathbf{x}_n)$.
- We would normally classify an instance as positive if $g_\theta(\mathbf{x}_n) >= 0$.
- However, we can achieve different tradeoffs between the true/false positives by introducing a threshold parameters τ and considering an instance to be positive if $g_\theta(\mathbf{x}_n) >= \tau$.
- This allows us to specify a TPR and FPR for every value of τ.

Performance Measures for Binary Classification

- True Positive Rate (TPR): The fraction of true positives to total positives instances.

$$TPR(\tau) = \frac{\sum_{n=1}^{N} [y_n = 1][g_\theta(\mathbf{x}_n) \geq \tau]}{\sum_{n=1}^{N} [y_n = 1]}$$

- False Positive Rate (FPR): The fraction of false positives to total negative instances.

$$FPR(\tau) = \frac{\sum_{n=1}^{N} [y_n \neq 1][g_\theta(\mathbf{x}_n) \geq \tau]}{\sum_{n=1}^{N} [y_n \neq 1]}$$

- We obtain a receiver operating characteristic (ROC) curve by sweeping the value of τ and plotting $TPR(\tau)$ vs $FPR(\tau)$.

Performance Measures for Classification

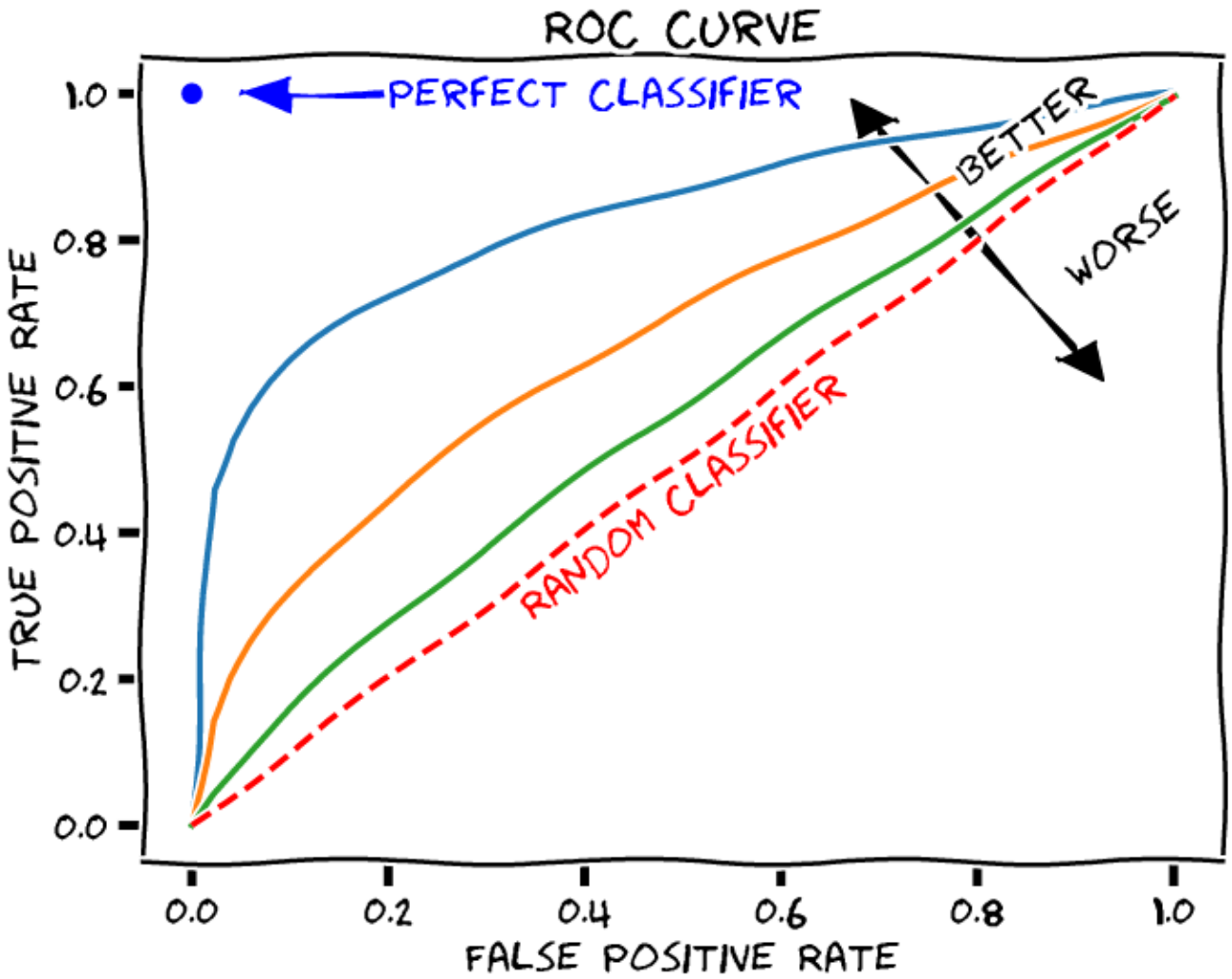


Image Credit: Martin Thoma

Performance Measures for Classification

- We can summarize an ROC curve using the area under the curve.
- This measure is referred to as AUC.
- Random guessing will yield an AUC of 0.5 regardless of class balance.
- The maximum possible AUC achieved by a classifier with a TPR of 1 and FPR of 0 is 1.

NLL, LL

- Negative Log Likelihood (NLL): Negative of average of log probability of test set labels.

$$NLL = -\frac{1}{N}\sum_{n=1}^{N} \log P_\theta(Y = y_n | \mathbf{X} = \mathbf{x}_n)$$

- Log Likelihood (LL): Average of log probability of test set labels.

$$LL = \frac{1}{N}\sum_{n=1}^{N} \log P_\theta(Y = y_n | \mathbf{X} = \mathbf{x}_n)$$

Weighted Measures

- Class-Weighted Negative Log Likelihood (NLLw): Negative of class-weighted average of log probability of test set labels.

$$NLLw = -\frac{\sum_{n=1}^{N} w_{y_n} \log P_\theta(Y = y_n | \mathbf{X} = \mathbf{x}_n)}{\sum_{n'=1}^{N} w_{y_{n'}}}$$

- Expected Misclassification Cost (EMC):

$$EMC = \frac{1}{N} \sum_{n=1}^{N} \sum_{y'} P_\theta(Y = y' | \mathbf{X} = \mathbf{x}_n) C[y_n, y']$$

Metric-Based

- Mean Squared Error: $MSE = \frac{1}{N}\sum_{n=1}^{N}(y_n - f_\theta(\mathbf{x}_n))^2$
- Root Mean Squared Error: $RMSE = \sqrt{\frac{1}{N}\sum_{n=1}^{N}(y_n - f_\theta(\mathbf{x}_n))^2}$
- Mean Absolute Error: $MAE = \frac{1}{N}\sum_{n=1}^{N}|y_n - f_\theta(\mathbf{x}_n)|$

Statistical

- Similar to class imbalance, regression models have a scale issue when interpreting results.
- One way to fix this problem is to consider the relative performance of a model compared to a baseline approach like predicting the mean of the target values $\bar{y}$.
- The coefficient of determination, fraction of explained variation and R^2 statistic all refer to the same measure:

$$R^2 = 1 - \frac{\sum_{n=1}^{N}(y_n - f_\theta(\mathbf{x}_n))^2}{\sum_{n=1}^{N}(y_n - \bar{y})^2}$$

Probabilistic

- Negative Log Likelihood:

$$NLL = -\frac{1}{N}\sum_{n=1}^{N} \log p_\theta(Y = y_n | \mathbf{X} = \mathbf{x}_n)$$

- Log Likelihood:

$$LL = \frac{1}{N}\sum_{n=1}^{N} \log p_\theta(Y = y_n | \mathbf{X} = \mathbf{x}_n)$$

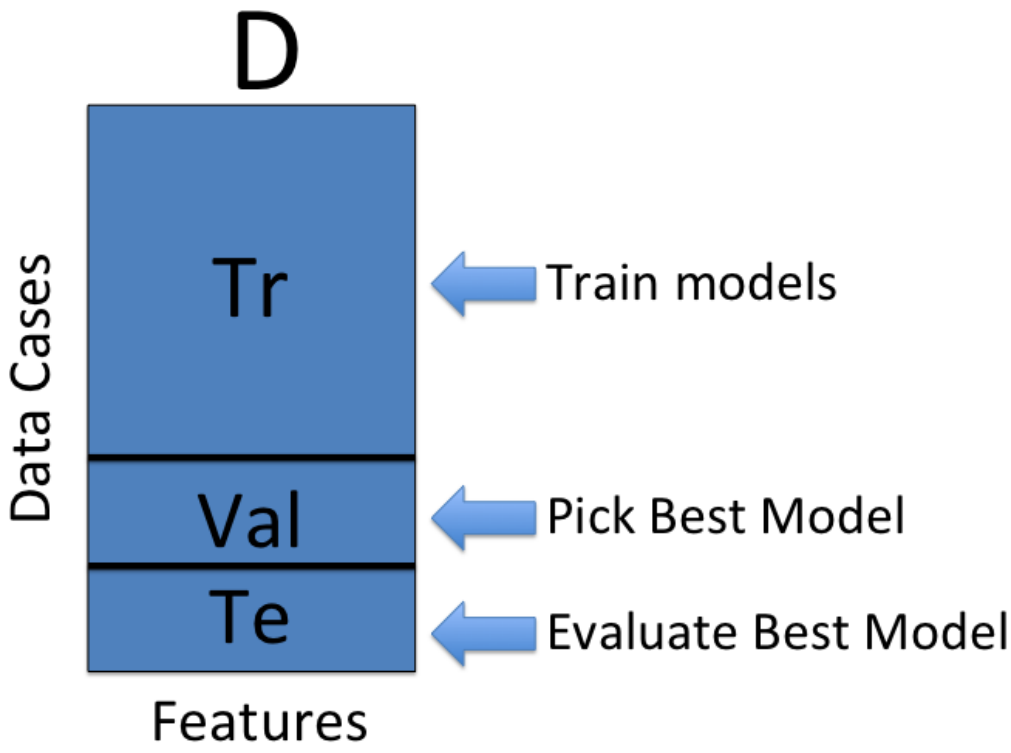
Experiment Design Principles

- Standard machine learning experiments aim to estimate generalization performance (performance on future data).
- Essentially all machine learning models include model complexity (regularization) hyper-parameters that require some form of validation-set method to select.
- More generally, we are often interested in comparing a proposed model to several baseline or state-of-the-art model to determine which method has the best generalization performance.
- You **must** apply hyper-parameter selection methods to **all** models that you compare if you want to draw conclusions about the relative performance of models. It is never acceptable to compare to a baseline model using its default regularization hyper-parameter values on a novel task.

Experiment Recipe 1: Train-Validation-Test

- Given a data set D, we randomly partition the data cases into a training set (Tr), a validation set (V), and a test set (Te). Typical splits are 60/20/20, 80/10/10, etc.
- Models M_i are learned on Tr for each choice of hyperparameters H_i
- The validation performance Val_i of each model M_i is evaluated on V.
- The hyperparameters H_* with the lowest value of Val_i are selected.
- The model can then be re-trained using these hyperparameters on $Tr + V$, yielding a final model M_*
- Generalization performance is estimated by evaluating the performance of M_* on the test data Te.

Example: Train-Validation-Test



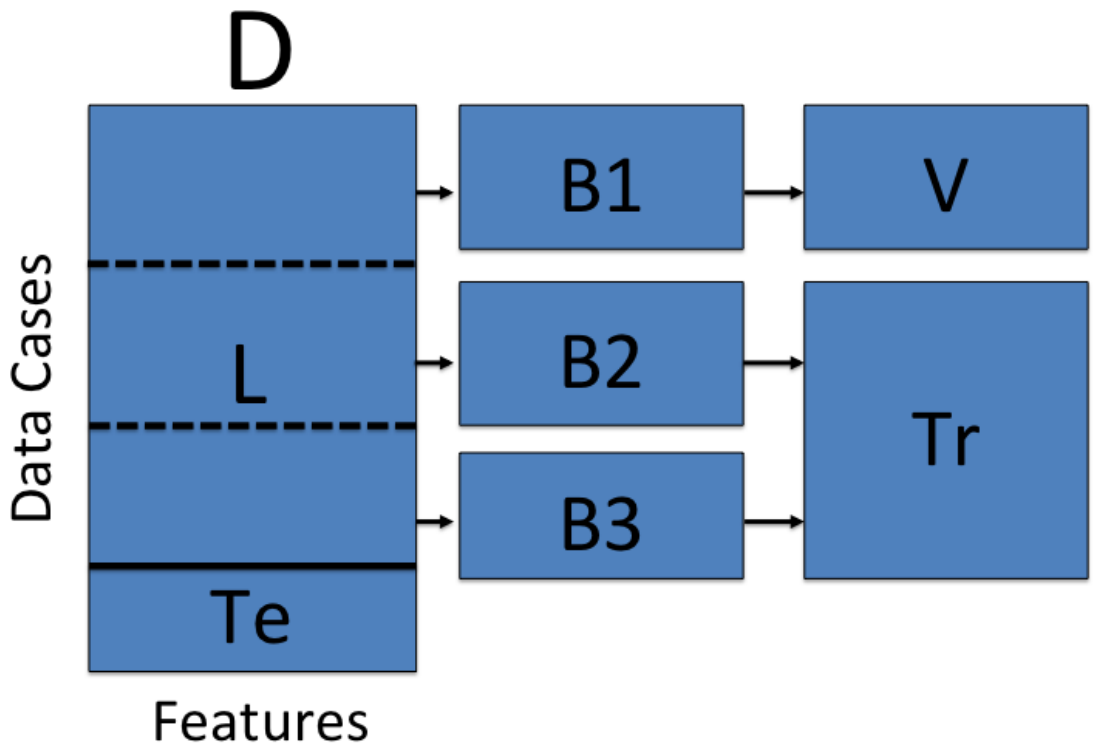
Note that the order of the data cases needs to be randomly shuffled before partitioning D.

Experiment Recipe 2: Crossvalidation-Test

- Randomly partition D into a learning set L and a test set Te (typically 50/50, 80/20, etc).
- We next randomly partition L into a set of K blocks $B_1, ..., B_K$.
- For each crossvalidation fold $k = 1, ..., K$:
  - Let $V = B_k$ and $Tr = L \setminus B_k$ (the remaining $K - 1$ blocks).
  - Learn M_{ik} on Tr for each choice of hyperparameters H_i.
  - Compute performance Val_{ik} of M_{ik} on V.
- Select hyperparameters H_* minimizing $\frac{1}{K} \sum_{k=1}^{K} Val_{ik}$.
- Re-train model on L using these hyperparameters, yielding final model M_*.
- Estimate generalization performance by evaluating error/accuracy of M_* on Te.

Example: 3-Fold Cross Validation and Test

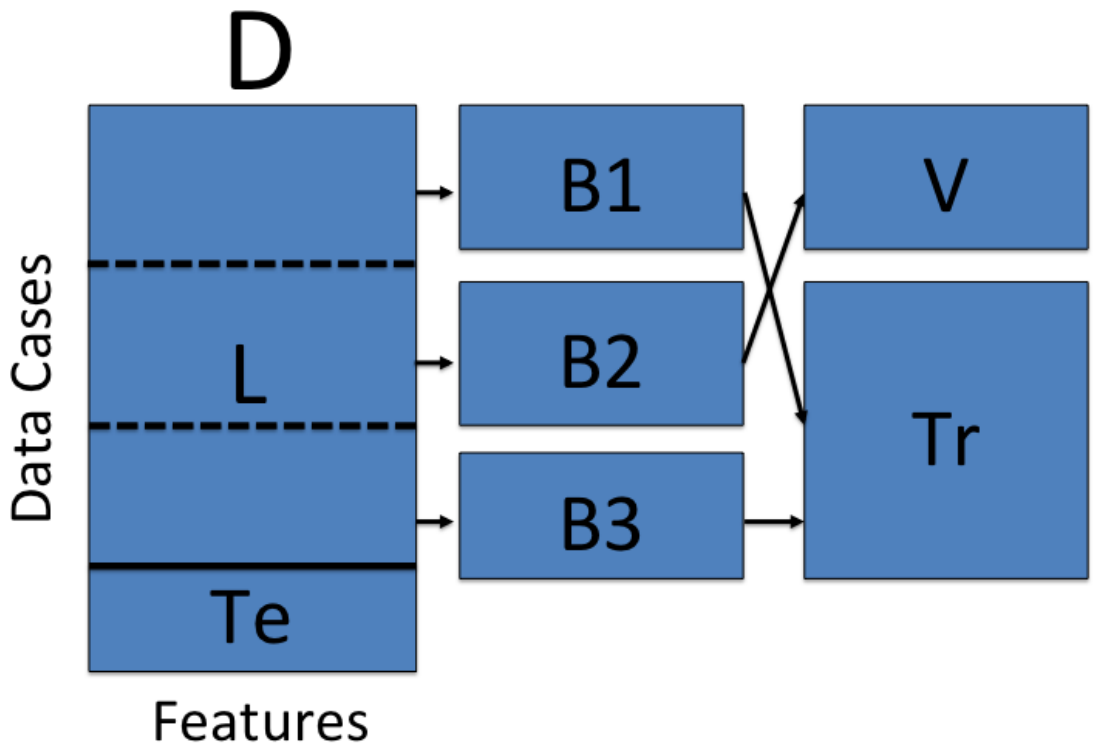
First Cross Validation Fold



Note that the order of the data cases needs to be randomly shuffled before partitioning D into L and Te.

Example: 3-Fold Cross Validation and Test

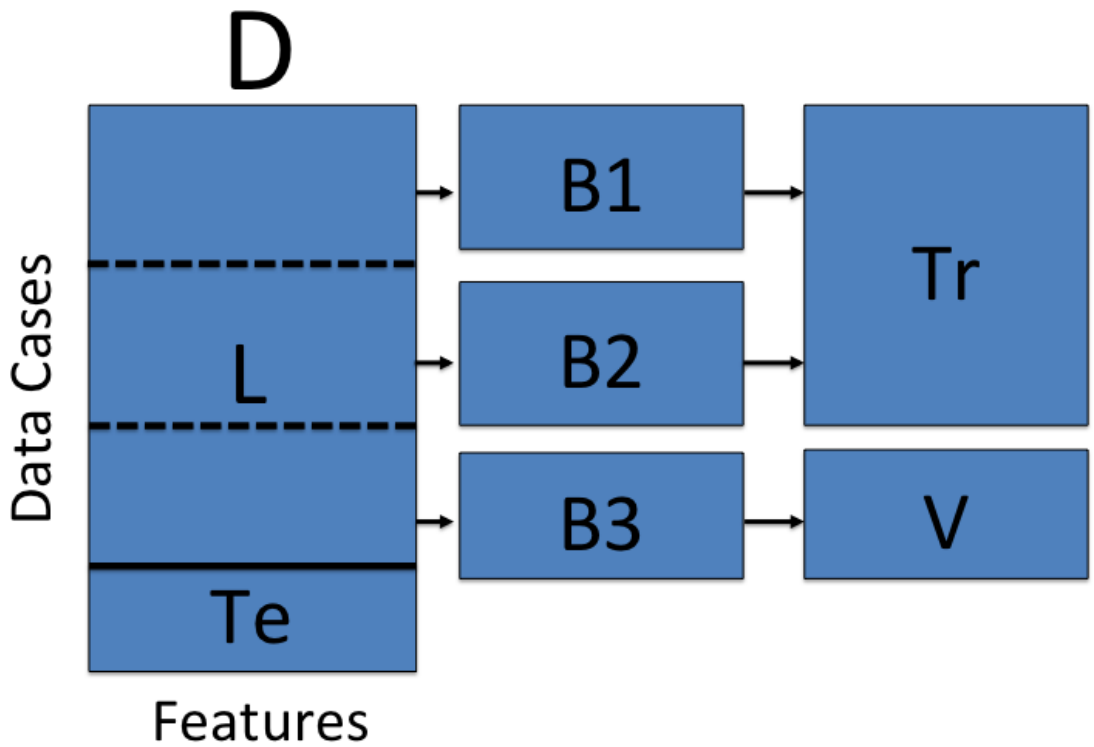
Second Cross Validation Fold



Note that the order of the data cases needs to be randomly shuffled before partitioning D into L and Te.

Example: 3-Fold Cross Validation and Test

Third Cross Validation Fold



Note that the order of the data cases needs to be randomly shuffled before partitioning D into L and Te.

Experiment Recipe 3: Crossvalidation-Crossvalidation

- Randomly partition data set D into a set of J blocks $C_1, ..., C_J$.
- For $j = 1, ..., J$:
  - Let $Te_j = C_j$ and $L_j = D \setminus C_j$
  - Partition L_j into a set of K blocks $B_1, ..., B_K$.
  - For $k = 1, ..., K$:
    - Let $V = B_k$ and $Tr = L \setminus B_k$.
    - Learn M_{ik} on Tr for each choice of hyperparameters H_i.
    - Compute error Val_{ik} of M_{ik} on V.
  - Select hyperparameters H_{*j} minimizing $\frac{1}{K} \sum_{k=1}^{K} Val_{ik}$ and re-train model on L_j using these hyperparameters, yielding model M_{*j}.
  - Compute Err_j by evaluating M_{*j} on Te_j.
- Estimate generalization error using $\frac{1}{J} \sum_{j=1}^{J} Err_j$

Experiment Design Trade-Offs

- In cases where the data has a benchmark split into a training set and a test set, we can use Recipes 1 or 2 by preserving the given test set and splitting the given training set into train and validation sets as needed.
- In cases where there is relatively little data, using a single held out test set will have high variance. In these cases, Recipe 3 often provides a better estimate of generalization error, but has much higher computational cost. However, it enables statistical significance testing.
- Choosing larger K in cross validation will reduce variance but increases computational costs. $K = 3, 5, 10$ are common choices for cross validation. $K = N$, also known as Leave-one-out cross validation is also popular when feasible.

Best Practices for Running Experiments

- You should make sure that you fix the random seeds used to partition data and initialize models so all of your results are reproducible (at least by you).
- For models with multiple local optima, you may want to average or max out over initializations to reduce variability.
- You should save the learned model for each fold and hyper-parameter value along with all performance measures of interest in case you need them later.
- You should make sure that the optimal values of all hyper-parameters selected do not fall at the endpoints of the ranges you tested.
- It's a good idea to make plots (for yourself) showing learning curves (train/validation performance vs training iterations) as well as cross-validation plots showing training and validation scores vs hyper-parameter values.